CSE-Tutor: Teaching a Small Language Model to Solve and Explain Computer Science Problems

Satya Dedeepya

Course project, Deep Learning · github.com/satyadedeepya/slm_dl_project

***Abstract*—Small language models can run on a student's own laptop, for free and without sending data anywhere. However, they often get the numerical problems of computer-science exams wrong. We fine-tuned Qwen2.5-1.5B-Instruct with LoRA to work as a tutor for six subjects: data structures, programming, databases (DBMS), operating systems (OS), computer networks (CN) and deep learning (DL). The key idea is *verified training data*: we wrote 41 small programs that create practice problems and compute the correct answer by running the real algorithm, so every answer is guaranteed to be right. All training and testing ran on a laptop with an integrated Intel GPU. On unseen test questions, the fine-tuned model is clearly more accurate than the original model: SQL writing rises from 50.5% to 60.0%, DBMS problems from 15% to 30% and networking problems from 1% to 8%, and an experiment without the verified data shows that this data causes the gains. We then added deep learning and two training stages that teach the model to explain like a teacher. A blind comparison by a judge model shows that the final model solves numerical problems better than the original model (13 wins to 5) and is much more accurate (DBMS 15→35%, DL 2→13%), while the original model still explains open concept questions slightly better.**

***Index Terms*—small language models, fine-tuning, LoRA, synthetic data, computer science education.**

I. Introduction

Students preparing for university exams, GATE and job interviews need a helper that can explain ideas, solve numerical problems step by step and write correct code and SQL. Large online models such as ChatGPT can do this, but they need the internet, may cost money and send the student's data to a company. A small model with 1.5 billion parameters can run on an ordinary laptop instead. The problem is that, out of the box, such a model gets many exam problems wrong. For example, it copies the *shape* of a CPU-scheduling solution but fills it with wrong waiting times.

The usual way to improve a small model is to fine-tune it on questions and answers written by a bigger model [9], [10]. But those answers are not checked, so the small model can learn the big model's mistakes. We take a different approach for the parts of the syllabus that are calculations. We write **generators**: short programs that make a random problem and then *compute* its answer by actually running the algorithm (for example, simulating the CPU scheduler or the CRC division). Every training answer they produce is correct by construction.

We study four questions:

- **RQ1:** How much does fine-tuning improve a small model compared with simply prompting it?
- **RQ2:** Is the verified data really the reason for the improvement on numerical problems?
- **RQ3:** Can we add a new subject (deep learning) later without the model forgetting what it learned?
- **RQ4:** Does the fine-tuned model also *explain* better than the original model, and if not, can we fix that?

Our main contributions are: (1) 41 tested problem generators for OS, CN, DBMS and DL; (2) a controlled experiment showing that the verified data causes the gains; (3) careful testing on seven unseen test sets with statistical tests; (4) a blind, fair way to compare explanations, and two training stages that improve them; and (5) a chat app that runs fully on a laptop.

II. Related Work

LoRA [1] fine-tunes a model by training small extra matrices instead of all its weights, and QLoRA [2] also compresses the base model to 4 bits. We use LoRA on Qwen2.5-1.5B-Instruct [3]. Other work has fine-tuned models for programming education [5], for data-structures problems [6] and as local teaching assistants [7], [8]. CS-Bench [4] is a benchmark for computer-science knowledge, and we use it in our tests. Self-Instruct [9] and GLAN [10] build training data with language models, and Orca [11] shows that small models learn a reasoning style by copying a teacher's step-by-step explanations. LIMA [12] argues that fine-tuning mainly teaches the *format* of answers, which matches what we found. Strong models can act as judges of answer quality, but they tend to prefer the first or the longer answer [13], so we judge each pair twice in swapped order. Training on a new task can make a model forget old skills [14]; we reduce this by mixing old examples into every new stage.

III. Methodology

Fig. 1 gives an overview. We collect training data from four kinds of sources, each with its own quality check, train five versions of the model one after another, test each version, and serve all of them in one chat app.

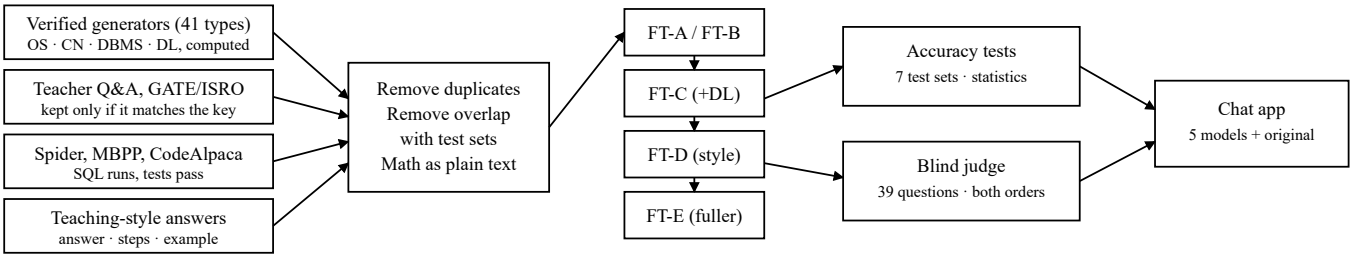


Fig. 1. Overview: data sources and their quality checks, the five training stages (each later stage continues from the previous one), the two kinds of tests, and the chat app.

A. Verified problem generators

Each generator picks random numbers, runs the real algorithm and writes a step-by-step solution that ends with a line "Final answer: …". It also stores the answer in a form a program can check automatically (Table I). Questions state any rules that could change the answer, such as how ties are broken. Test problems use different number ranges from training problems, so the model cannot memorise them. We checked the generators with automatic tests: against worked examples from standard textbooks, and, for deep learning, by computing the same answers again with PyTorch.

TABLE I
VERIFIED PROBLEM GENERATORS

Subject	Problem types	Train / test
OS	CPU scheduling (6 algorithms), page replacement, Banker's algorithm, disk scheduling	900 / 100
CN	subnetting, CRC, checksum, delays, sliding window	900 / 100
DBMS	attribute closure, candidate keys, normal forms, lossless join	600 / 100
DL	convolution and pooling sizes, parameter counts, forward pass, softmax loss, gradient descent, backpropagation, attention, precision/recall	600 / 100

EXAMPLE GENERATED PROBLEM

"A convolutional layer receives an input of size 227×227×6. It has 20 filters of size 11×11, stride 2 and padding 5. Find the output size and the number of trainable parameters."

Computed answer: (227 + 2×5 − 11)/2 + 1 = 114, so the output is 114×114×20; parameters = (11×11×6 + 1)×20 = 14,540.

B. The training data

The main training set has 9,335 examples (Table II). Besides 2,400 generated problems, it contains GATE and ISRO past exam questions, short concept questions and answers, questions based on free textbooks, SQL questions from Spider [15] and Python tasks from MBPP [16] and CodeAlpaca. Where a bigger "teacher" model wrote explanations, it did so *without seeing the official answer*, and we kept an explanation only if the teacher's answer matched the official key (990 of 1,154 GATE/ISRO questions). We removed duplicate examples and any training example that overlapped with a test question.

TABLE II
MAIN TRAINING SET

Source	Examples	Quality check
Verified generators (OS, CN, DBMS)	2,400	answer computed by code
GATE / ISRO past questions	978	teacher agrees with key
Concept questions (teacher)	1,458	agrees with key
Textbook questions	1,170	answer is in the passage
Spider (SQL)	1,335	SQL query runs
MBPP / CodeAlpaca (Python)	311 / 1,184	tests pass / code runs
General examples	499	length check

C. Five training stages

We used LoRA with rank 16, which trains only 18.5 million of the 1.5 billion parameters (1.18%). Each stage ran for one pass over its data (one epoch) with an effective batch of 32 examples. The laptop's integrated Intel GPU does not support 4-bit models reliably, so we kept the base model in 16-bit precision. Table III lists the five stages:

- **FT-A and FT-B** are a matched pair. FT-B uses the full training set; FT-A uses the same set *without* the generated problems. Comparing them shows what the verified data adds.
- **FT-C** continues from FT-B and adds deep learning: 600 verified DL problems and 307 DL concept answers. It also includes 1,200 old examples ("replay") so the model does not forget earlier subjects.
- **FT-D and FT-E** teach the explanation style. We noticed that the teacher's concept answers in the training data were short (median 57–91 words), while the original model writes about 290 words. So we wrote new answers for about 1,300 concept questions in a fixed teaching format: a direct answer, numbered key points, a worked example and an "In short" summary, with all maths written as plain text. FT-D learns these answers (median 164 words); FT-E learns fuller versions of 250–350 words. Both also replay verified problems to keep the accuracy.

TABLE III
THE FIVE TRAINING STAGES (EACH LATER STAGE CONTINUES FROM THE STAGE IN THE "STARTS FROM" COLUMN)

Stage	Starts from	Training data	Examples	Training time
FT-A	original	main set without generated problems (comparison run)	6,588	3.7 h
FT-B	original	full main set	8,868	4.8 h
FT-C	FT-B	deep-learning problems and concepts + 1,200 old examples	2,107	70 min
FT-D	FT-C	1,192 teaching-style answers + 1,300 verified and code examples	2,492	96 min
FT-E	FT-D	1,254 fuller teaching-style answers + 550 verified and code examples	1,804	81 min

D. How we tested the models

Accuracy. We tested every model on questions it had never seen: 400 CS-Bench questions [4], 100 new generated problems each for OS, CN, DBMS and DL, 200 MBPP coding tasks (the code must pass its tests), 200 Spider SQL questions (the query must return the right result) and 56 recent GATE questions. We compared the fine-tuned models with the original model, both without examples (B0) and with three worked examples in the prompt (B1). To check whether a difference is real or just luck, we used the McNemar test [17], which compares two models on the same questions; a difference with $p < 0.05$ is called significant.

Explanation quality. Accuracy only checks the final answer, not whether the explanation helps a student. So we also ran a **blind comparison** on 39 unseen questions: 19 numerical problems (with their correct solutions) and 20 concept questions such as "What is CNN?". A separate judge model (Gemma 4 31B) sees two answers labelled only "A" and "B", scores each for correctness, steps and clarity from 1 to 5, and picks the better one. It is told not to prefer an answer just because it is longer. Each pair is judged twice with A and B swapped, and a model wins a question only if it wins both times.

TABLE IV
TEST SETS (NONE OF THESE QUESTIONS WERE USED IN TRAINING)

Test set	Questions	How an answer is scored
CS-Bench (DSA, OS, CN)	400	correct option chosen
Generated OS / CN / DBMS / DL	100 each	all final numbers correct
MBPP (Python)	200	code passes the tests
Spider (SQL)	200	query gives the right result
GATE 2021–26	56	correct option chosen
Explanation set	39	blind judge, both orders

E. The chat app

To make the tutor usable, we built a chat app that runs on the same laptop. A small Python server loads the original model once and attaches all five trained versions as separate LoRA adapters, so the user can switch between them, or switch them off to see the original model, without reloading anything. Answers appear word by word. Any mathematics the model writes in LaTeX code is shown as plain readable text, for example "$z = \Sigma\, w_i x_i + b$", so students never see raw symbols. A second app creates a fresh generated problem, lets the model solve it and marks the answer right or wrong using the computed solution.

IV. Results

TABLE V
ACCURACY (%) ON UNSEEN TEST QUESTIONS. * = SIGNIFICANTLY BETTER THAN THE ORIGINAL MODEL B0 ($P < 0.05$); – = NOT TESTED

Test	Questions	B0 original	B1 3 examples	FT-A	FT-B	FT-C	FT-D	FT-E
CS-Bench (concepts)	400	51.2	**58.0***	53.8	55.2	–	–	–
OS problems	100	4	4	3	5	–	–	–
CN problems	100	1	2	0	8*	7	**12***	11*
DBMS problems	100	15	12	17	30*	31*	24	**35***
DL problems	100	2	–	–	3	5	**21***	13*
MBPP (Python code)	200	41.5	41.5	41.0	**42.5**	40.0	39.5	40.0
Spider (SQL)	200	50.5	–	54.5	**60.0***	59.5*	58.5*	59.0*
GATE 2021–26	56	33.9	–	**37.5**	33.9	–	–	–

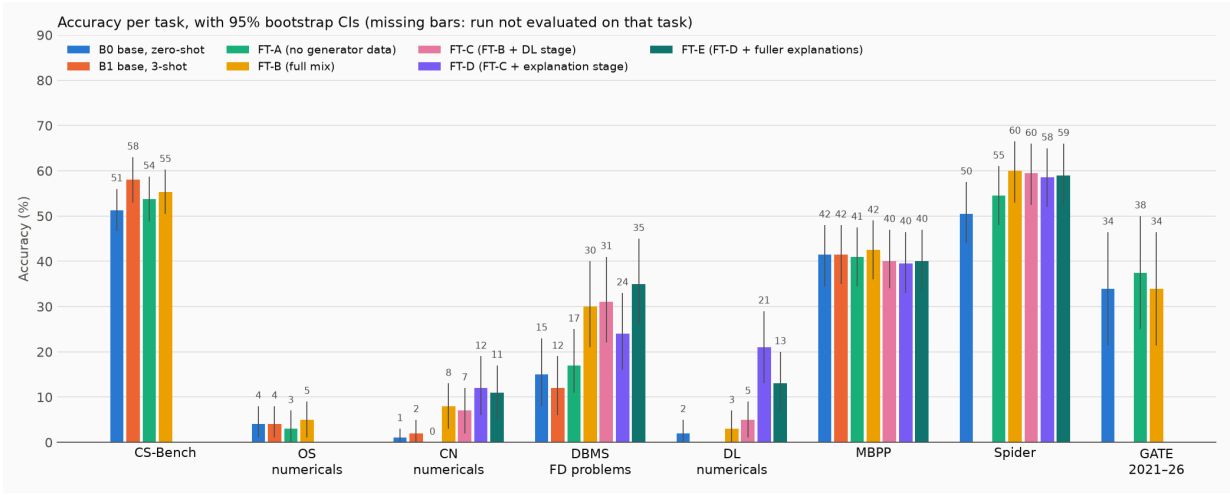


Fig. 2. Accuracy of every model on every test. The thin black lines show the 95% confidence interval: when the lines of two bars overlap a lot, the difference may be due to chance.

A. RQ1: Does fine-tuning help?

Yes, on the tasks where we had strong data (Table V and Fig. 2). FT-B is significantly better than the original model at writing SQL (50.5→60.0%), at DBMS problems (15→30%) and at networking problems (1→8%). Coding (MBPP) and GATE scores did not change, so the model did not lose general skills. On CS-Bench, which tests broad knowledge, fine-tuning helped only a little (+4 points), and about half of that came from the model writing its answer in the expected format more often. Here, simply giving the original model three examples (B1) worked slightly better. OS problems stayed hard for every model.

B. RQ2: Is the verified data the reason?

Yes. FT-A was trained in exactly the same way as FT-B but without the generated problems, and it stayed at the original model's level on every numerical test (for example 0% on networking problems, while FT-B reached 8%; $p = 0.008$). On the other tests the two models are about equal. So the verified data is what teaches the procedures, such as finding attribute closures and subnet ranges. One caution: FT-B also trained for 35% more steps than FT-A.

C. RQ3: Adding deep learning without forgetting

FT-C learned deep learning in 70 minutes of extra training. Getting *every* number of a DL answer right is hard, so we also measured the share of correct numbers in each answer: it rose from 15.8% (FT-B) to 26.0% (FT-C), a significant gain. The biggest improvements were in convolution sizes (22→47%) and softmax loss (0→56%). Thanks to the replayed old examples, the scores on DBMS, networking, SQL and coding stayed the same. The later stages FT-D and FT-E improved DL further, to 21% and 13% fully correct answers.

D. RQ4: Does the model explain well?

Table VI shows the blind comparison against the original model. FT-C solved numerical problems better (14 wins to 4) but lost on concept questions (6 to 13). Its answers were short, a median of 79 words against 286 for the original model, because it had learned from short training answers. The judge gave FT-C 2.7 out of 5 for explaining the steps of concept answers, against 4.2 for the original model.

TABLE VI
BLIND COMPARISON AGAINST THE ORIGINAL MODEL: WINS–LOSSES–TIES (* = SIGNIFICANT, SIGN TEST P < 0.05)

Model	Numerical (19)	Concepts (20)	Answer length
Original (B0)	–	–	286 words
FT-C	**14–4–1***	6–13–1	79 words
FT-D	**13–4–2***	8–12–0	102 words
FT-E	**13–5–1**	6–11–3	194 words

TABLE VII
JUDGE SCORES FOR CONCEPT QUESTIONS (1–5): FINE-TUNED MODEL / ORIGINAL MODEL

Model	Correctness	Steps	Clarity
FT-C	3.5 / 3.4	2.7 / 4.2	3.4 / 4.3
FT-D	3.5 / 3.4	3.2 / 4.0	3.9 / 4.3
FT-E	3.1 / 3.3	3.6 / 4.0	3.8 / 4.2

The teaching-style stages fixed part of this problem. FT-D and FT-E kept the lead on numerical problems, and FT-E's answers became much fuller (194 words). Its score for explaining concept steps rose to 3.6 (original: 4.0). However, the original model still won slightly more concept questions (11 to 6, with 3 ties); this difference is not significant. FT-E is also the most accurate model on DBMS problems (35%) and is significantly better than the original model on DBMS, DL, networking and SQL, with coding unchanged.

The judge's comments show *why* the fine-tuned model wins numerical questions. On the convolution problem from Section III-A, both models gave a wrong final number. The original model used the wrong formula, ignoring the padding. FT-E used the correct formula and the correct values, but made one subtraction mistake:

FT-E's answer (shortened)

```
Output size = (N + 2P − K) / S + 1
            = (227 + 2·5 − 11) / 2 + 1 = 219 / 2
+ 1 = 110
Parameters  = (11·11·6 + 1) × 20 = 14740
```

(227 + 10 − 11 is 226, which gives 114 and 14,540.) The judge preferred FT-E because it "uses the correct formulas and fails only on basic arithmetic."

E. Where the model still makes mistakes

To understand the remaining errors, we read 50 wrong answers of FT-B by hand and gave each one main cause (Table VIII). None of them failed because of the answer format: the model fully learned how to present a solution. More than half were errors in *carrying out* a long procedure. For example, when finding an attribute closure, the model applied one dependency and then stopped too early; in FCFS scheduling it ordered processes by their number instead of their arrival time; in LRU page replacement it drew a table with more frames than allowed. The model knows the method, but it does not track every step reliably. SQL errors were mostly about using a column from the wrong table, even though the table definitions were given.

TABLE VIII
CAUSES OF 50 WRONG ANSWERS (FT-B)

Cause	Share	Example
Procedure not carried out correctly	52%	closure stops after one rule
Missing knowledge	16%	mixes up deadlock avoidance and prevention
Wrong table or column in SQL	12%	column taken from another table
Arithmetic or units	8%	0.96 ms written as 9.6 ms
Misread the task	6%	returns a number instead of true/false
Answer contradicts its own reasoning	4%	argues for A, answers B
Unclear question	2%	–

F. Examples the final model gets right

Table IX shows some unseen test questions that FT-E answered correctly and the original model answered wrongly. They cover the procedures the verified data teaches: closures, subnetting, network delays and layer sizes.

TABLE IX
UNSEEN TEST QUESTIONS SOLVED BY FT-E BUT NOT BY THE ORIGINAL MODEL

Subject	Question (shortened)	Correct answer
DBMS	R(A,B,C,D,E), FDs {CD→A, BD→C, AC→E, A→D, C→BE}: find C+	{B, C, E}
DBMS	FDs {DE→BC, CD→BE, D→C} on R(A,B,C,D,E): candidate keys	{A, D}
CN	Host 214.247.173.4/27: network, broadcast, usable hosts	.0, .31, 30 hosts
CN	3000-byte packet, 3 store-and-forward links of 8 Mbps and 4000 km: total delay	3×(3 + 20) = 69 ms
CN	1200-byte frames, 3 Mbps, 15 ms one-way delay: minimum window, sequence bits	11 frames; 4 (GBN), 5 (SR)
DL	Dense network 12 → 100 → 10 with biases: number of parameters	2,310

V. What We Learned

Checked answers beat more answers. The generated problems were only about a quarter of the training data, yet they were the only part that improved numerical accuracy. Teacher-written answers, textbook questions and exam questions did not have this effect. For calculation-heavy topics, a few thousand *correct* worked solutions are worth more than many unchecked ones.

The model copies the style of its training answers. When the training answers were short, the model's answers became short, even for open questions where the original model gave long, clear explanations. When we trained on longer teaching-style answers, the model's answers grew from 79 to 194 words and followed the same structure. This means that good explanations have to be designed into the training data; they do not appear by themselves.

Accuracy and explanation are different goals. A model can become more accurate while explaining less, as FT-C did. Measuring only accuracy would have hidden this. The blind comparison made it visible and showed whether each fix helped.

Each stage has trade-offs. FT-D is best on deep-learning problems, while FT-E is best on DBMS problems and gives fuller explanations. Because each stage was trained only once, part of these differences may be due to chance, so the chat app keeps both versions available.

A laptop is enough. All five training stages together took about 12.6 hours on an integrated GPU. The main practical problems were memory limits, missing 4-bit support and the laptop going to sleep during long runs, which we solved with gradient accumulation, saved checkpoints and a keep-awake setting.

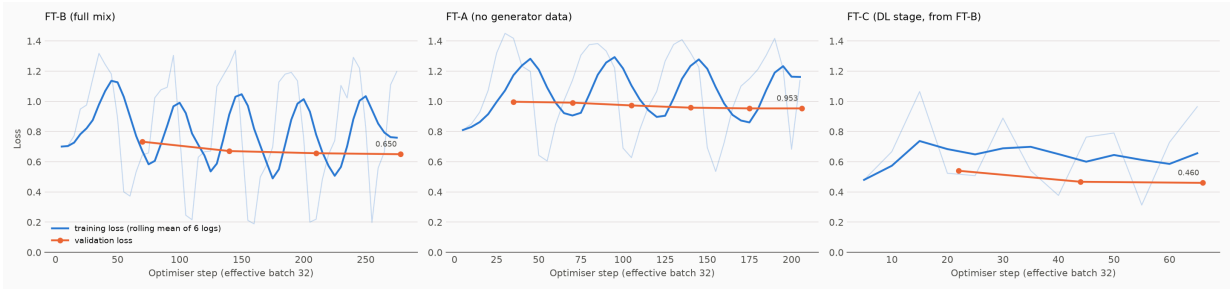


Fig. 3. Training loss (smoothed) and validation loss of the first training stages. Lower is better; the curves were still falling slowly at the end, so longer training may help.

VI. Conclusion

We showed that a small 1.5-billion-parameter model, fine-tuned on a laptop, can become a much better computer-science tutor. The most important ingredient is training data whose answers are **computed by code**: it taught the model procedures that neither prompting nor AI-written answers taught, and a controlled experiment confirmed this. New subjects can be added in about an hour of extra training without forgetting old ones. We also found that **the model copies the length and style of its training answers**. Short answers made it accurate but terse, and longer teaching-style answers brought its explanations close to, but not yet above, the original model's.

Limitations. Our study has some limits that readers should keep in mind:

- Each stage was trained only once and for one epoch, so some differences between stages may be due to chance.
- FT-B trained for more steps than FT-A, which slightly weakens the comparison between them.
- The explanation test uses one AI judge and only 39 questions; a rating by human teachers would make it stronger.
- The GATE test has only 56 questions, and the later models were tested on five of the eight test sets to save time.
- Some training answers were written by a larger AI model, which can make mistakes even after our checks.

Future work. We see four clear next steps: let the model call a calculator or run code for arithmetic and long simulations [19]; train for more epochs with several random seeds; use preference training such as DPO [18] on the answer pairs the judge already compared; and test the tutor with real students in a course.

Responsible use. Because the tutor runs on the student's own laptop, no data leaves the device. Its answers can still be wrong, especially in long calculations, so the app says this clearly and the practice mode always shows the verified answer.

References

1. E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," arXiv:2106.09685, 2021.
2. T. Dettmers *et al.*, "QLoRA: Efficient finetuning of quantized LLMs," in *NeurIPS*, 2023.
3. Qwen Team, "Qwen2.5 technical report," arXiv:2412.15115, 2024.
4. X. Song *et al.*, "CS-Bench: A comprehensive benchmark for large language models towards computer science mastery," arXiv:2406.08587, 2024.
5. "Supervised fine-tuning LLMs to behave as pedagogical agents in programming education," arXiv:2502.20527, 2025.
6. "Fine-tuning of large language models for solving DSA problems using QLoRA," IEEE, doc. 10969969.
7. "AI-TA: Towards an intelligent question-answer teaching assistant using open-source LLMs," arXiv:2311.02775, 2023.
8. "Small models, big support: A local LLM framework for educator-centric content creation and assessment," arXiv:2506.05925, 2025.
9. Y. Wang *et al.*, "Self-Instruct: Aligning language models with self-generated instructions," arXiv:2212.10560, 2022.
10. H. Li *et al.*, "Synthetic data (almost) from scratch: Generalized instruction tuning for language models," arXiv:2402.13064, 2024.
11. S. Mukherjee *et al.*, "Orca: Progressive learning from complex explanation traces of GPT-4," arXiv:2306.02707, 2023.
12. C. Zhou *et al.*, "LIMA: Less is more for alignment," in *NeurIPS*, 2023.
13. L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *NeurIPS Datasets and Benchmarks*, 2023.
14. "Analyzing and reducing catastrophic forgetting in parameter efficient tuning," arXiv:2402.18865, 2024.
15. T. Yu *et al.*, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in *EMNLP*, 2018.
16. J. Austin *et al.*, "Program synthesis with large language models," arXiv:2108.07732, 2021.
17. Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
18. R. Rafailov *et al.*, "Direct preference optimization: Your language model is secretly a reward model," in *NeurIPS*, 2023.
19. L. Gao *et al.*, "PAL: Program-aided language models," in *ICML*, 2023.